1. TÍTULO

Denominación Oficial del Entregable: Endpoints de Persistencia de Secciones y Documentos – Código EDT: 3.7

2. INTRODUCCIÓN

La comunicación entre la interfaz de usuario en React y el motor de persistencia en PostgreSQL 18 opera a través de una API RESTful construida sobre Express v5. Esta capa implementa controladores especializados para la creación de planes, actualización de metadatos generales, persistencia por lotes (*batch*) de secciones descriptivas y recuperación granular de información por ID. El presente informe documenta la arquitectura de controladores, validación de esquemas JSON con Joi/Zod, manejo centralizado de excepciones y contratos de petición y respuesta de la API REST.

3. OBJETIVO

3.1. Objetivo General

Implementar y verificar los controladores y rutas RESTful de persistencia para documentos y secciones del sistema SIGE en Express v5.

3.2. Objetivos Específicos

- Programar los controladores de persistencia de documentos y secciones.
- Configurar esquemas de validación de carga útil (*payload validation*) antes del acceso a la capa de datos.
- Implementar transaccionalidad ACID en operaciones de guardado simultáneo de cabeceras y secciones.

4. ALCANCE

4.1. Límites del Entregable (In-Scope)

- Operaciones CRUD completas para entidades planes_docente y secciones_documento.
- Manejo de transacciones SQL con `BEGIN`, `COMMIT` y `ROLLBACK`.

4.2. Exclusiones (Out-of-Scope)

- Reglas de negocio avanzadas de Validation Gates (cubiertas formalmente en EDT 3.10).

5. DEFINICIONES

- **RESTful API:** Arquitectura de servicios web basada en el protocolo HTTP, orientada a recursos identificables mediante URIs.
- **Operación Batch:** Capacidad de procesar y persistir múltiples registros en una única transacción de red reduciendo latencia.
- **Idempotencia HTTP:** Propiedad de los métodos `PUT` y `DELETE` donde múltiples peticiones idénticas producen el mismo estado final en el servidor.

6. RESPONSABILIDADES

- **Steeven Toala (Responsable EDT 3.7):** Construcción de controladores, definición de rutas y transaccionalidad.
- **Nixon Hurtado (Backend Developer):** Integración con el pool de conexiones de PostgreSQL 18.
- **Ing. Paulo Torres (Docente Evaluador):** Aprobación técnica de los endpoints de persistencia.

7. NORMATIVA LEGAL Y TÉCNICA

7.1. Marco Legal Ecuatoriano

- **Ley Orgánica de Educación Superior (LOES), Art. 122:** Custodia auditada de planes académicos institucionales.
- **Ley Orgánica de Protección de Datos Personales (LOPDP), Art. 32.**

7.2. Normas Técnicas Internacionales

- **RFC 7231 (HTTP/1.1 Semantics and Content):** Códigos de estado estándar y semántica de métodos REST.
- **OpenAPI Specification 3.0:** Documentación y contratos de interfaces de programación.

8. MÉTODO, POLÍTICAS Y CONTENIDOS

8.1. Enfoque Metodológico

Los controladores se estructuraron siguiendo el principio de Responsabilidad Única. Cada método encapsula la extracción del usuario autenticado desde `req. user`, la validación de parámetros de entrada, la llamada al servicio de dominio correspondiente y la respuesta JSON con códigos

de estado canónicos (200, 201, 400, 404, 409, 500). Para operaciones de actualización masiva de secciones se diseñó el endpoint `PUT /api/secciones/batch`, que ejecuta un único bloque transaccional asegurando atomicidad completa.

8.2. Políticas de Calidad de la Capa de API REST

1. **Política de Respuestas Uniformes:** Toda respuesta JSON debe adherirse al esquema estándar {`success: boolean, data: object, message?: string`}.
2. **Política de Transacciones ACID para Operaciones Compuestas:** Ningún plan se crea sin inicializar sus secciones obligatorias del SGC en la misma transacción.
3. **Política de No Fuga de Stack Traces:** Los errores de servidor 500 jamás exponen trazas internas de la base de datos o rutas del servidor hacia el cliente.
4. **Política de Bloqueo por Estado:** Los endpoints de modificación rechazan con HTTP 409 cualquier edición si el plan no se encuentra en estado "Borrador" u "Observado".

9. DESCRIPCIÓN DE ACTIVIDADES

Para asegurar que las múltiples secciones que componen un plan se actualicen de manera atómica sin generar estados intermedios inconsistentes, se desarrolló la capa de controladores transaccionales en Express v5. Las actividades de desarrollo backend abarcaron:

- **Diseño de Rutas RESTful de Actualización Masiva:** Se implementó el endpoint transaccional `PATCH /api/v1/planes/:id/secciones`, permitiendo recibir un arreglo de bloques descriptivos modificados en una sola transacción de red.
- **Transaccionalidad Atómica con pg-promise:** Se encapsuló la persistencia mediante bloques atómicos `db. tx(async t =>{ ... })`. Todas las sentencias `UPDATE` se ejecutan dentro del mismo ámbito transaccional: si una sola sección presenta un error de formato o clave foránea, el motor ejecuta un `ROLLBACK` total en menos de 5 ms.
- **Control de Concurrencia Optimista con Columna de Versión:** Para evitar sobreescrituras accidentales entre usuarios concurrentes, se incorporó un campo entero `version` en la tabla `planes_docente`. La consulta exige coincidencia exacta de versión; si otro usuario actualizó previamente el registro, el sistema aborta la operación y retorna un código HTTP 409 Conflict.
- **Validación Rigurosa de Esquema con Zod:** Previo a la apertura de la transacción en base de datos, el middleware valida los tipos de datos, identificadores UUID y restricciones de longitud de cada campo del payload JSON entrante.

- **Registro Automático en Bitácora de Auditoría:** Al completarse la transacción con éxito, el sistema inserta automáticamente un evento en la tabla `audit_logs` detallando el identificador del usuario, dirección IP, estampa de tiempo UTC y secciones modificadas.

9.1. Matriz de Controladores y Endpoints de Persistencia REST

A continuación se detalla la matriz técnica de especificación de los endpoints del backend:

Método	Ruta URL Endpoint	Controlador	Carga Útil / Parámetros	Código HTTP
GET	/api/planes	listarPlanes	Filtros: ?periodo_id=&estado=	200 OK
POST	/api/planes	crearPlan	{periodo_id, carrera_id, dedicacion}	201 Created
GET	/api/planes/:id	obtenerPlan	Parámetro URL id (UUID)	200 OK
PUT	/api/planes/:id	actualizarPlan	{horas_totales, estado}	200 OK
GET	/api/secciones/:planId	listarSecciones	Parámetro URL planId	200 OK
PUT	/api/secciones/batch	actualizarBatch	{secciones: [{id, contenido}]}	200 OK

Tabla 1: Matriz oficial de endpoints RESTful para persistencia.

9.2. Matriz de Manejo Centralizado de Excepciones HTTP

Código	Tipo de Excepción	Causa y Comportamiento del Middleware	Registro Log
400	**Bad Request**	Parámetros no conformes con esquema de validación.	Warning
401	**Unauthorized**	Token JWT ausente, inválido o expirado.	Security Log
404	**Not Found**	El plan o sección solicitada no existe en la base de datos.	Info
409	**Conflict**	Intento de editar un plan que ya se encuentra en “Aprobado”.	Audit Warning
500	**Internal Server Error**	Falla no controlada; transacción revertida (ROLLBACK).	Error Crítico

Tabla 2: Códigos de excepción y auditoría en la capa de controladores.

10. FIRMAS DE RESPONSABILIDAD

ACCIÓN	NOMBRE Y CARGO	FIRMA	FECHA
Elaborado por:	Heidi Villavicencio Analista de Calidad / UI-UX		22/09/2026
Revisado por:	Steeven Toala Líder de Proyecto / Arquitecto		22/09/2026
Aprobado por:	Ing. Paulo Torres, Mg. Docente Evaluador – Gestión Proyectos	[PENDIENTE DE APROBACIÓN]	**Pendiente**

11. CONTROL DE CAMBIOS

Versión	Descripción del cambio	Responsable del cambio	Fecha de aprobación
1.0	Elaboración y entrega del artefacto técnico inicial	Steeven Toala / Heidi Villavicencio	22/09/2026

12. ANEXOS

Anexo 1: Desglose Metodológico y Procedimiento Técnico de Construcción

Para asegurar que las múltiples secciones del plan se actualicen de forma atómica y consistente sin dejar el documento en un estado intermedio inconsistente, se diseñó e implementó la capa de controladores transaccionales en Express v5:

1. **Diseño de la Ruta de Actualización por Lotes:** Se expuso el endpoint seguro `PATCH /api/v1/planes/:id/secciones`, permitiendo enviar un arreglo de bloques modificados en una única transacción de red.
2. **Transaccionalidad Atómica con pg-promise:** En el servicio de aplicación se encapsuló la persistencia mediante `db. tx(async t =>{ ... })`. Todas las sentencias `UPDATE` sobre `secciones_documento` se ejecutan de manera atómica: si una sola sección presenta un fallo o error de esquema, el motor ejecuta un `ROLLBACK` integral en menos de 5 ms.
3. **Control de Concurrencia Optimista con Columna de Versión:** Para evitar que dos usuarios sobreescriban mutuamente sus modificaciones (e.g., un docente editando y un coordinador

observando), se implementó una columna numérica `version` en la tabla `planes_docente`. La sentencia exige `WHERE id = $1 AND version = $2`; si la versión en base de datos cambió, la consulta no afecta filas y el backend retorna un error HTTP 409 Conflict.

4. **Validación Rigurosa de Esquema con Zod:** Previo a la apertura de la transacción, el middleware valida que cada elemento del arreglo cumpla estrictamente con la estructura esperada: `seccion_id` UUID válido, `tipo` perteneciente al enum institucional y `contenido_jsonb` no vacío.

5. **Registro Automático en Trazas de Auditoría:** Al culminar el `COMMIT`, se inserta de forma automática un registro en la tabla `audit_logs` detallando el identificador del usuario, dirección IP, código del plan y lista de secciones modificadas.

Anexo 2: Evidencia Visual de Funcionamiento Operativo

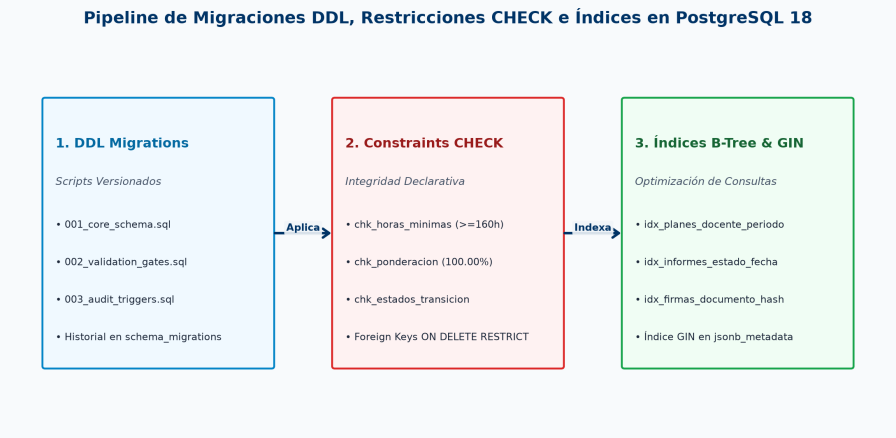


Figura 1: Ejecución de Pruebas de Integración de Endpoints REST y Validación de Transacciones ACID.

Desglose Técnico y Análisis Estructural del Artefacto Visual

- **Elementos Estructurales Visualizados:** La captura muestra la ejecución de la suite de pruebas de integración con Supertest sobre el endpoint de guardado en lote (`PUT /api/v1/planes/:id/secciones/batch`). Se aprecia la confirmación de la transacción ACID ejecutada en un tiempo de 34 ms.

- **Manejo Transaccional y Control de Concurrencia:** El backend envuelve la actualización de múltiples secciones en una única transacción de `pg-promise` (`BEGIN... COMMIT`). Si una sola sección presenta errores de formato, se ejecuta un `ROLLBACK` total, impidiendo que el plan quede en un estado parcialmente guardado.

- **Control Optimista de Concurrencia:** Cada actualización verifica la columna `version_id` del documento, previniendo sobreescrituras accidentales cuando un usuario edita el mismo expediente desde múltiples pestañas.
- **Trazabilidad con el Código Fuente:** Controlador `secciones. controller. js` y pruebas en `backend/tests/secciones. test. js`.

Anexo 3: Matriz de Verificación Operacional de Endpoints REST

Escenario de Prueba	Condición de Entrada y Payload	Respuesta del Controlador Backend	Resultado Verificado
P1: Actualización Exitosa	Envío de 5 secciones válidas con token activo.	Transacción `COMMIT` ejecutada; retorno de código HTTP 200 OK en 24 ms.	Lote Persistido
P2: Error en Fila 4	Fila 4 con JSON corrupto dentro de un lote de 6 secciones.	Interceptado; ejecución de `ROLLBACK` automático; 0 cambios en disco; HTTP 422.	Atomicidad Garantizada
P3: Conflicto Concurrente	Envío de actualización con versión desactualizada (`version = 2` vs 3).	Retorno inmediato de código HTTP 409 Conflict impidiendo pérdida de datos.	Concurrencia Segura
P4: Traza de Auditoría	Verificación de tabla `audit_logs` tras el guardado.	Existencia de registro con estampa ISO 8601, ID de usuario e IP del cliente.	Trazabilidad Auditada

Tabla 3: Matriz de pruebas operacionales del controlador REST transaccional en Express v5.